Web Application Security Risks

A Technical Brief Based on the OWASP Top 10

Name : Pranay Vinod Kadam

Intern Id : 2404

1. Purpose and Method

This brief uses the OWASP Top 10:2025, the eighth edition of the list and the first update since 2021, as a starting point for understanding common web application security risks. OWASP describes the list as an awareness document, not a testing checklist: its ten categories group 248 CWEs and draw on contributed data plus a practitioner survey. This brief examines six categories in depth: A01 Broken Access Control, A02 Security Misconfiguration, A03 Software Supply Chain Failures, A04 Cryptographic Failures, A05 Injection and A07 Authentication Failures. For each, it explains how the weakness arises, the impact, and the controls that prevent it. The emphasis throughout is defensive: design decisions and engineering practices that remove a weakness class rather than patch individual instances.

2. Risk Analysis

01: Broken Access Control

How it arises: The application fails to enforce, on the server, what each user may see or do. Typical causes are authorization checks applied only in the user interface, direct object references (such as `/invoices/1042`) that are never checked against the caller's ownership, missing checks on administrative or API endpoints, and permissive CORS settings. It remains the top-ranked category.

Impact: Unauthorized disclosure, modification or deletion of other users' data, and privilege escalation to administrative functions. Because the flaw lives in business logic, scanners often miss it.

Defenses: Deny by default; enforce authorization on the server for every request in one central layer; verify object ownership on every data access; use unguessable identifiers as defense in depth, not as the control itself; log and alert on access-control failures; write automated tests that attempt cross-user and cross-role access.

02: Security Misconfiguration

How it arises: Insecure defaults or inconsistent settings across the application, framework, web server, cloud services and containers: debug modes left on, default credentials, verbose error pages, unnecessary services, public storage buckets, missing security headers. It rose from fifth to second place, reflecting how much security now depends on configuration.

Impact: Information leakage that aids further attacks, unintended public exposure of data and, in severe cases, full environment compromise.

Defenses: Build from a hardened, minimal baseline; manage configuration as reviewed, version-controlled code so environments stay identical; remove unused features; apply security headers such as Content-Security-Policy and Strict-Transport-Security; return generic errors to users; run automated configuration scanning in CI and production.

03: Software Supply Chain Failures

How it arises: Applications depend on third-party libraries, build tools, package registries and CI/CD pipelines. Risk appears through known-vulnerable or abandoned components, compromised or typosquatted packages, unpinned versions and weakly protected build systems. This category broadens the earlier "Vulnerable and Outdated Components."

Impact: Malicious or vulnerable code runs with the application's full privileges, potentially affecting every customer who receives the build.

Defenses: Maintain a software bill of materials (SBOM) covering direct and transitive dependencies; pin versions and verify integrity with lockfiles and hashes; run software composition analysis and act on advisories promptly; use a trusted internal registry or proxy; restrict and audit changes to build pipelines; require review of dependency changes; follow NIST SSDF practices.

04: Cryptographic Failures

How it arises: Sensitive data is sent or stored without adequate protection: plaintext HTTP, outdated protocols or ciphers, weak password hashing, hard-coded or poorly rotated keys, or homemade cryptography.

Impact: Exposure of credentials and personal or financial data, with regulatory and reputational consequences.

Defenses: Classify data and encrypt it in transit (TLS 1.2 or higher, HSTS) and at rest; use vetted libraries and current algorithms, never custom schemes; hash passwords with a slow, salted algorithm such as Argon2id, scrypt or bcrypt; keep keys in a managed secrets or key-management service with rotation; do not collect or retain data you do not need.

05: Injection

How it arises: Untrusted input is concatenated into an interpreter's command or query (SQL, OS shell, LDAP, template engines) or into a browser page (cross-site scripting), so data is executed as instructions. It has fallen to fifth place, helped by safer frameworks, but remains common in custom code. Prompt injection in LLM applications is a distinct problem covered by the separate OWASP LLM Top 10.

Impact: Data theft or tampering, authentication bypass, remote code execution, and session hijacking through XSS.

Defenses: Use parameterized queries or safe ORM APIs; avoid invoking shells with user data; validate input on the server with allow-lists as a secondary control; encode output for its context and rely on auto-escaping templates; apply a Content-Security-Policy; run the database account with least privilege; test with SAST and DAST.

06: Authentication Failures

How it arises: Weak or absent protection against credential stuffing and brute force, permitted weak passwords, missing multi-factor authentication (MFA), predictable or long-lived session identifiers, session IDs exposed in URLs, and insecure recovery flows.

Impact: Account takeover, which gives an attacker the victim's full legitimate access.

Defenses: Offer or require MFA, preferably phishing-resistant methods such as passkeys; follow NIST SP 800-63B guidance (length over composition rules, screening against breached passwords); rate-limit and monitor login attempts; issue random session IDs, regenerate them at login, set `Secure`, `HttpOnly` and `SameSite` cookie attributes, and invalidate sessions on logout; return uniform messages in login and recovery flows.

3. Cross-Cutting Secure-Design Practices

Three themes recur across the categories. First, **centralize security decisions** (authorization, input handling, cryptography) in well-tested shared components, so fixes apply everywhere. Second, **apply least privilege** to users, services, database accounts and pipelines. Third, **make controls verifiable**: threat-model during design (OWASP A06 Insecure Design), automate tests and scans in CI, and log security events with alerting (A09). OWASP's Application Security Verification Standard (ASVS) provides a testable set of requirements for turning these practices into acceptance criteria.

4. Risk-to-Mitigation Table

Risk (OWASP 2025)	Typical root cause	Primary impact	Key preventive controls	Verification
01 Broken Access Control	Missing server-side checks; unchecked object references	Data exposure; privilege escalation	Deny by default; central authorization; ownership checks; access-failure logging	Cross-user and cross-role automated tests
02 Security Misconfiguration	Insecure defaults; inconsistent environments; debug left on	Information leakage; exposed services and data	Hardened baselines; infrastructure as code; security headers; generic errors	Configuration scanning in CI and production
03 Software Supply Chain Failures	Unvetted or vulnerable dependencies; weak build pipeline	Malicious code runs with app privileges	SBOM; pinned and hashed dependencies; trusted registry; pipeline access control	Composition analysis; dependency review gates
04 Cryptographic Failures	No TLS; weak algorithms; poor key management	Credential and personal data exposure	TLS plus HSTS; vetted libraries; Argon2id, scrypt or bcrypt; managed keys	TLS configuration testing; secrets scanning
05 Injection	Untrusted input mixed with commands or markup	Data theft; code execution; XSS	Parameterized queries; context-aware output encoding; CSP; least-privilege DB account	SAST, DAST, targeted input testing
06 Authentication Failures	Weak passwords; no MFA; poor session handling	Account takeover	MFA or passkeys; breached-password screening; rate limiting; secure session cookies	Login abuse tests; session-handling review

5. References

1. OWASP Foundation. OWASP Top 10:2025 (including the Introduction). https://owasp.org/Top10/2025/
2. OWASP Cheat Sheet Series (Authorization, Password Storage, SQL Injection Prevention, Cross-Site Scripting Prevention, Session Management). https://cheatsheetseries.owasp.org/
3. OWASP Application Security Verification Standard (ASVS). https://owasp.org/www-project-application-security-verification-standard/
4. OWASP Top 10 for LLM Applications (for prompt injection). https://genai.owasp.org/
5. NIST SP 800-63B, Digital Identity Guidelines: Authentication and Lifecycle Management. https://pages.nist.gov/800-63-4/
6. NIST SP 800-218, Secure Software Development Framework (SSDF). https://csrc.nist.gov/pubs/sp/800/218/final
7. MITRE. CWE Top 25 Most Dangerous Software Weaknesses. https://cwe.mitre.org/top25/